REACT · LECTURE SERIES

React Hooks

A complete, in-depth, simple-English guide to every core Hook —
useState, useEffect, useContext, useRef, useMemo, useCallback, useReducer & custom Hooks.

Functional components • State & lifecycle • Best practices

LECTURE ROADMAP

What We'll Cover Today

S

useState

Store & update data in a component

E

useEffect

Run side effects (fetch, timers, subscriptions)

C

useContext

Share data without passing props manually

R

useRef

Persist values & reach real DOM elements

M

useMemo

Cache expensive calculations

CB

useCallback

Cache function definitions

RD

useReducer

Manage complex state logic

CH

Custom Hooks

Build your own reusable logic

THE BIG IDEA

What Exactly Is a Hook?

A Hook is just a special **JavaScript function** — always starting with the word **"use"** — that lets a simple function component "hook into" React features like state and lifecycle, which used to be available only in class components.

In one line:

> *"Hooks let function components remember things and react to changes — without ever writing a class."*

Why they were introduced (2019, React 16.8):

- Reuse stateful logic between components easily
- Avoid deeply nested "wrapper hell" from old patterns
- Keep related code together instead of split across lifecycle methods

BEFORE — Class Component

```
class Box extends React.Component {
  state = { count: 0 };
  render() {
    return <p>{this.state.count}</p>;
  }
}
```

AFTER — Function + Hook

```
function Box() {
  const [count] = useState(0);
  return <p>{count}</p>;
}
```

BEFORE YOU START

The Two Golden Rules of Hooks

1 Only call Hooks at the top level

Never put a Hook inside a loop, condition, or nested function. Always call Hooks in the exact same order on every single render, so React can correctly match state to the right Hook.

✓ DO

```
if (x) { ... }
const [v] = useState(0);
```

✗ DON'T

```
if (x) {
  const [v] = useState(0);
}
```

2 Only call Hooks from React functions

Call Hooks only from React function components, or from your own Custom Hooks. Never call them from a regular JavaScript function, a class, or an event handler directly.

✓ DO

```
function Profile() {
  const [n] = useState('');
}
```

✗ DON'T

```
function helper() {
  const [n] = useState('');
}
```

S

HOOK 1

useState

Give a function component its own memory.

useState lets a component store a value that can change over time — like a counter, form input, or toggle — and re-render automatically whenever that value updates.

KEY POINTS

- useState(initialValue) returns an array: [value, setterFunction]
- Calling the setter schedules a re-render with the new value
- State updates do NOT merge automatically like class state did
- Each useState call manages one independent piece of state
- Never mutate state directly — always call the setter function

```
import { useState } from 'react';


function Counter() {
  const [count, setCount] = useState(0);


  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Simple way to remember: “box” = current value, “button” = the way to change it.

E

HOOK 2

useEffect

Run code in response to renders — side effects.

useEffect lets you run code that reaches outside React — fetching data, setting a timer, subscribing to an event — after the component renders, and clean it up when needed.

KEY POINTS

- Runs after the browser has painted the screen
- [] (empty array) = run once, only on mount
- [value] = re-run only when that value changes
- No array at all = run after every single render
- Return a function from it to clean up (timers, listeners)

```
import { useEffect, useState } from 'react';

function Clock() {

  const [time, setTime] = useState(new Date());

  useEffect(() => {

    const id = setInterval(() =>

      setTime(new Date()), 1000);

    return () => clearInterval(id); // cleanup

  }, []); // dependency array

  return <p>{time.toLocaleTimeString()}</p>;
}
```

The dependency array is React's way of asking: “when should this effect run again?”

C

HOOK 3

useContext

Share data across many components — no prop drilling.

useContext lets any component read a value from a shared “Context” directly, skipping the need to pass props down manually through every level in between.

KEY POINTS

- Removes the pain of passing props through many layers
- A Provider wraps components and supplies the shared value
- Any descendant can read it with useContext(TheContext)
- Great for themes, logged-in user, language, or app settings
- Updating the Provider's value re-renders every consumer

```
const ThemeContext = React.createContext('light');

function App() {

  return (

    <ThemeContext.Provider value='dark'>

      <Toolbar />

    </ThemeContext.Provider>

  );
}

function Toolbar() {

  const theme = useContext(ThemeContext);

  return <button className={theme}>Save</button>;
}
```

Think of Context like a radio station: any component can “tune in” without wiring.

HOOK 4

useRef

Keep a value around without causing a re-render.

useRef gives you a mutable “box” (.current) that survives across renders. Changing it never triggers a re-render, and it's also the standard way to directly access a DOM element.

KEY POINTS

- useRef(initialValue) returns an object: { current: value }
- Updating ref.current does NOT cause a re-render
- Attach it to a JSX tag with ref={...} to grab the DOM node
- Perfect for timers IDs, previous values, or focus control
- Different from state: refs are for values React shouldn't watch

```
import { useRef } from 'react';

function TextInput() {

  const inputRef = useRef(null);

  const focusInput = () => {

    inputRef.current.focus(); // DOM node

  };

  return (

    <>
      <input ref={inputRef} />

      <button onClick={focusInput}>Focus</button>
    </>
  );
}
```

Rule of thumb: if changing it should update the UI → useState. If not → useRef.

M

HOOK 5

useMemo

Skip expensive recalculations on every render.

useMemo remembers (“memoizes”) the result of a calculation, and only re-computes it when one of its listed dependencies actually changes, saving unnecessary work.

KEY POINTS

- Caches a computed VALUE between renders
- Recalculates only when a dependency in the array changes
- Useful for sorting, filtering, or heavy math on big lists
- Not free — it has its own small cost, so don't overuse it
- Only reach for it after noticing an actual performance issue

```
import { useMemo } from 'react';

function ProductList({ items, filter }) {

  const visible = useMemo(() => {

    console.log('filtering...');

    return items.filter(i =>

      i.category === filter);

  }, [items, filter]); // recompute if changed

  return visible.map(i =>
    <li key={i.id}>{i.name}</li>);
}
```

Simple test: “Is this calculation slow AND does it run on every render?” If yes, memoize it.

CB

HOOK 6

useCallback

Cache a function so it isn't rebuilt every render.

useCallback works just like useMemo, but for FUNCTIONS instead of values. It returns the same function reference across renders unless its dependencies change.

KEY POINTS

- Every render normally creates a brand-new function — useCallback prevents that
- Most useful when passing a function to a memoized child (React.memo)
- Without it, that child re-renders even when nothing meaningful changed
- useMemo caches a VALUE; useCallback caches a FUNCTION
- useCallback(fn, deps) is really just useMemo(() => fn, deps)

```
import { useCallback, useState } from 'react';

function Parent() {

  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {

    setCount(c => c + 1);

  }, []); // same function every render

  return <FastButton onClick={handleClick} />;
}

const FastButton = React.memo((
  { onClick }) => (
  <button onClick={onClick}>+1</button>
));
```

Pair it with React.memo — caching a function alone doesn't help unless a child depends on it.

RD

HOOK 7

useReducer

A more organized way to manage complex state.

useReducer is an alternative to useState for state that's complex or involves multiple sub-values. You describe WHAT happened by dispatching an “action”, and a reducer function decides the next state.

KEY POINTS

- useReducer(reducerFn, initialState) returns [state, dispatch]
- dispatch(action) sends a plain object describing what happened
- The reducer is a pure function: (state, action) => newState
- Keeps state-update logic in one predictable, testable place
- Great choice when many actions can update related state fields

```
function reducer(state, action) {

  switch (action.type) {

    case 'inc': return { count: state.count + 1 };

    case 'dec': return { count: state.count - 1 };

    default: return state;
  } }

function Counter() {

  const [state, dispatch] =
    useReducer(reducer, { count: 0 });

  return (
    <button onClick={() => dispatch({type:'inc'})}>
      {state.count}
    </button>
  );
}
```

Same idea used by Redux! useReducer is essentially Redux's core pattern, built into React.

CH

HOOK 8

Custom Hooks

Package your own reusable logic as a Hook.

A Custom Hook is simply a JavaScript function whose name starts with “use”, and that calls other Hooks inside it. It lets you extract and reuse stateful logic across components.

KEY POINTS

- Solves the old problem of duplicating logic across components
- Built entirely from built-in Hooks like useState and useEffect
- Each component using it gets its own, fully independent state
- Naming it “useSomething” lets React enforce the Rules of Hooks
- Common examples: useFetch, useForm, useLocalStorage, useDebounce

```
// A reusable hook for window width

function useWindowWidth() {

  const [width, setWidth] =
    useState(window.innerWidth);

  useEffect(() => {

    const onResize = () =>
      setWidth(window.innerWidth);

    window.addEventListener('resize', onResize);

    return () =>
      window.removeEventListener('resize', onResize);

  }, []);
  return width;
}

const width = useWindowWidth(); // any component
```

Custom Hooks share LOGIC, not state — each caller gets a fresh, separate copy.

QUICK REFERENCE

Hooks Cheat Sheet

Hook	Purpose	Use It When…
`useState`	Store a piece of data that changes	You need simple local state
`useEffect`	Run side effects after render	Fetching data, timers, subscriptions
`useContext`	Read shared data from a Provider	Avoiding prop drilling
`useRef`	Persist a value / access a DOM node	Focus control, timers, previous value
`useMemo`	Cache a computed value	An expensive calculation is re-running
`useCallback`	Cache a function reference	Passing callbacks to memoized children
`useReducer`	Manage complex, multi-part state	Many related actions update one state



Thank You

Questions about any Hook? Let's discuss.

`useState` · `useEffect` · `useContext` · `useRef` · `useMemo` · `useCallback` · `useReducer` · `Custom Hooks`